



AI FULL-STACK & DEPLOYMENT ENGINEER · TRACK 2.B · SESSION 3 · DAY 8

Multimodal Input & RAG-Backed Features

Let your app SEE documents and photos — and answer from YOUR documents, with sources and honest confidence

Claude Vision · GPT-4o · Gemini 2.5 · Embeddings · Chroma · Citations · Confidence badges

Session 8 of 10 · 2 hours · concept 45 min + hands-on lab 60 min + review 15 min



What we will do in the next 2 hours

45 min



Concepts & walkthrough

Multimodal input (images + documents into the model), why RAG, the 8-step RAG pipeline, citations and confidence UX.

60 min



Hands-on lab

Ingest policy documents into a vector store, build a grounded /ask endpoint, render inline citations + a confidence badge in the UI.

15 min



Review & wrap-up

Demo each other's apps with an out-of-corpus question — does it refuse honestly? Faithfulness asserts recap.

Goal: leave today with an app that reads images and PDFs, answers ONLY from your documents, and shows its sources like an honest consultant.



THE PROBLEM

Real business inputs are messy. Plain LLM answers are risky.

Pain 1 — Input is not clean text

What actually arrives at an enterprise app:

- Scanned claim forms — tilted, stamped, handwritten corners
- 40-page policy PDFs with tables and annexures
- Photos of a damaged car taken in bad light
- Lab reports clicked on a phone camera

Pain 2 — Plain answers can't be trusted

What enterprise users actually need:

- A bare LLM invents clause numbers and cover limits — confidently. That is a compliance incident, not a feature.
- **Users need: answer + WHICH document/page it came from + how sure the system is.**
- “Answer + source + confidence” is the enterprise contract.



Desi analogy: A good doctor reads your reports, looks at the X-ray AND listens to you — together. A good consultant always says which page of the report his claim comes from. Today we build both habits into our app.



MULTIMODAL MODELS

The big three — what each can actually see



Claude (Sonnet / Opus)

Images + native PDF input

Careful, grounded document reading; strong at forms, tables and long policies; good at saying “unreadable” instead of guessing.



GPT-4o (OpenAI)

Images + audio in / out

Fast, strong general-purpose vision; huge ecosystem of examples; natural fit for voice + image chat products.



Gemini 2.5 (Google)

Images, PDF, audio, video

Very large context (about 1M tokens) — an entire policy book or a site-survey video in ONE request.

Sending images: base64 (data URL)

- Image bytes travel INSIDE the request — works for private files (claims, reports)
- Bigger payload; resize/compress before sending

Sending images: public URL

- You send only a link; the provider fetches the image
- **Image must be publicly reachable — almost never true for enterprise documents. Default to base64.**

Rule of thumb: enterprise documents are private → base64. Marketing images on your CDN → URL.



One message, two parts: the image AND the question

```
# vision.py — image + text in ONE message
# (OpenAI format, sent via LiteLLM from Session 7)
import base64, litellm

img = base64.b64encode(
    open("claim_form.jpg", "rb").read()).decode()

resp = litellm.completion(
    model="claude-sonnet-4-5", # or gpt-4o / gemini-2.5-pro
    messages=[{
        "role": "user",
        "content": [          # a LIST of parts, not a string
            {"type": "image_url",
             "image_url": {"url":
                f"data:image/jpeg;base64,{img}"}}},
            {"type": "text",
             "text": "What is the claim amount
                     on this form?"},
        ],
    },
)
print(resp.choices[0].message.content)
```

What changed vs text chat?

- content is now a list of typed parts — image_url + text
- Image goes as a data: URL — the base64 bytes ride inside
- Everything else — role, completion call, response shape — is identical

Why this matters

- Session 7's gateway means ONE code path for Claude, GPT-4o, Gemini — change the model string, nothing else.
- Order matters a little: image first, then the question — like showing the X-ray before asking the doctor.



DOCUMENTS

PDFs: send pages as images, or send the PDF itself

Road 1 — Pages as images

- Convert each page to a PNG (pdf2image / PyMuPDF), send as image parts
- **Works with EVERY vision model — the universal fallback**
- Model literally LOOKS at the page: stamps, signatures, layout, tables survive
- Costly for long PDFs — each page is an image's worth of tokens

Road 2 — Native PDF input

- Claude and Gemini accept the PDF file directly (base64 document part)
- Provider does the page handling — you send ONE part for the whole file
- **Best of both: text layer AND page visuals are read together**
- Check per-provider page limits before promising “any PDF”

Decision: scanned/stamped forms → pages-as-images (visual truth). Clean digital policies → native PDF where supported. Either way, today's RAG pipeline takes over after extraction.



Desi analogy: The doctor again: Road 1 is pinning each X-ray sheet on the lightbox one by one; Road 2 is handing over the whole file and letting the hospital sort the pages.



PATTERN

Photo in → validated JSON out (Session 6, now with eyes)

```
# extract.py – claim form photo to clean fields
from pydantic import BaseModel, ValidationError

class ClaimForm(BaseModel):          # Session 6 pattern
    policy_no: str
    claim_amount: float
    incident_date: str

PROMPT = """Extract from this form image. Reply ONLY
with JSON: policy_no, claim_amount, incident_date.
If any field is unreadable, use null."""

resp = litellm.completion(model=MODEL,
    messages=[{"role": "user", "content": [
        img_part(photo), {"type": "text",
            "text": PROMPT}]]])

try:
    claim = ClaimForm.model_validate_json(
        resp.choices[0].message.content)
except ValidationError:
    # bad photo or missing fields -> ask again
    return {"retry": "Photo unclear – please re-upload"}
```

The ask-again UX

- Blurry / cropped / dark photo? Don't guess — tell the user exactly what to fix: “amount corner is cut, re-click covering the full form”.
- **Pydantic failure = the trigger. Validation error → friendly retry message, not a stack trace.**
- Log retry rates — a rising rate means your upload UX needs work, not your prompt.



Desi analogy: A bank clerk returning a form: “aapka signature clear nahin hai, yahan dobara kijiye” — pointing at the exact box. Precise re-asks beat silent guessing.

Garbage photo in → validated refusal out. That IS the feature.



WHY RAG

Closed-book memory vs open-book answers

Closed-book exam

= plain LLM answering from memory

- Knowledge frozen at training time — knows nothing of YOUR policies, prices, circulars
- When unsure, it writes a fluent, confident, WRONG answer (hallucination)
- No way to verify — the answer cites nothing
- Updating knowledge = retraining. Slow, costly.

Open-book exam

= RAG: Retrieval-Augmented Generation

- We fetch the right pages from YOUR documents and paste them into the prompt
- **Model answers ONLY from what's on the desk — and cites the page**
- Update knowledge = drop in a new document. No retraining.
- “Not in the book” → say so. Honest refusal is allowed.

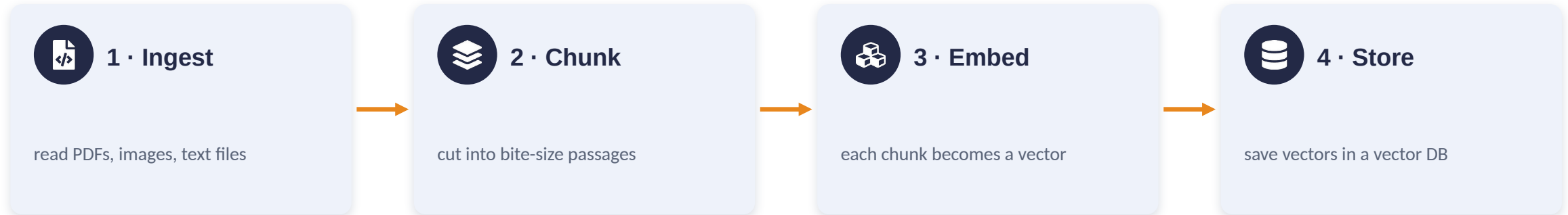


Desi analogy: Topper in a closed-book exam still can't answer from a textbook he never read. Give the same student an open book and a rule — “answer only from the book, quote the page” — and suddenly every answer is checkable.



Eight steps: four at build time, four at ask time

BUILD TIME — once per document



a question arrives...

ASK TIME — every user question



 **Desi analogy:** Before the exam you prepare the book: sticky bookmarks on every topic (ingest → store). In the exam hall you flip straight to the right bookmark and copy the answer **WITH** the page number (retrieve → cite).



Chunking, embeddings & where the vectors live

Chunking — cutting the documents

- Too BIG (2000+ tokens): retrieval gets fuzzy; irrelevant text drowns the answer
- Too SMALL (50 tokens): a clause is torn mid-sentence; meaning lost
- **Sweet spot: 300–500 tokens, respect sentence/paragraph boundaries**
- Overlap 10–15% so a clause split across two chunks survives in at least one

Vector store — pick by stage

Chroma

Embedded, zero setup, persists to a folder. Today's lab pick — prototype speed.

FAISS

In-memory library, blazing fast search. You handle persistence and serving yourself.

pgvector

Your existing Postgres learns vectors. One DB for app data + search — the production favourite.

Embeddings: all-MiniLM-L6-v2 (free, local, good enough for the lab) vs API embeddings like text-embedding-3-small (better quality, per-call cost). Rule: SAME embedding model for ingest and query — mixing models breaks retrieval silently.



Desi analogy: *Chunking is portion control at a wedding buffet: serve the sabzi in katoris, not the whole handi (too big) and not one spoon (too small) — with a little extra so nobody's plate misses the tadka.*



Ingestion: chunk → embed → store (once per document)

```
# ingest.py – build the open book (run once)
import chromadb
from chromadb.utils import embedding_functions

client = chromadb.PersistentClient("./vectordb")
emb = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2") # local, free
col = client.get_or_create_collection(
    "policies", embedding_function=emb)

def chunk(text, size=400, overlap=60):
    words = text.split()
    step = size - overlap # 15% overlap
    return [" ".join(words[i:i+size])
            for i in range(0, len(words), step)]

for doc_id, text in load_documents(): # your .txt files
    for i, ch in enumerate(chunk(text)):
        col.add(ids=[f"{doc_id}_c{i}"],
                documents=[ch],
                metadatas=[{"source": doc_id,
                           "chunk": i}])
```

Read it like a story

- PersistentClient = the library building (a folder on disk)
- embedding function = the librarian who files by MEANING, not alphabet
- ids like motor_c3 = the bookmark label — this becomes your citation!

Note for production

Re-ingesting? Delete or upsert old chunks first, or stale clauses haunt your answers. Ingestion is a pipeline, not a one-off script.



Query: retrieve top-k → grounded prompt → cited answer

```
# rag.py – answer every question from the book
res = col.query(query_texts=[question], n_results=4)

# number the chunks -> the model can point at them
context = "\n\n".join(
    f"[{i+1}] (id={cid}) {doc}"
    for i, (cid, doc) in enumerate(zip(
        res["ids"][0], res["documents"][0]))))

SYSTEM = """Answer ONLY from the numbered context.
After every claim, cite its source like [1] or [2].
If the answer is not in the context, reply exactly:
NOT_FOUND"""

resp = litellm.completion(model=MODEL, messages=[
    {"role": "system", "content": SYSTEM},
    {"role": "user",
     "content": f"Context:\n{context}\n\n"
                f"Question: {question}"}])
answer = resp.choices[0].message.content
```

Three load-bearing lines

- “Answer ONLY from context” — closes the model's own memory (the closed book stays shut)
- “cite [1] [2]” — numbered context blocks give it something to point AT
- “NOT_FOUND” — the escape hatch. Without it, the model fills silence with fiction.

Keep the scores!

`res["distances"]` holds how close each chunk was. We turn that into the confidence badge in two slides.



Rendering [1][2] as clickable, inspectable sources

```
# ui.py – Streamlit: markers -> chips + sources
import re, streamlit as st

ans, cites = ask_api(q)  # answer + citation list

# 1) make [1] [2] stand out as superscript chips
html = re.sub(r"\\(\\d+)\\",
              r"<sup style='background:#E8871E;color:white;
              padding:1px 5px;border-radius:8px'>\\1</sup>",
              ans)
st.markdown(html, unsafe_allow_html=True)

# 2) each citation opens its source passage
for c in cites:
    with st.expander(f"[{c['n']}] {c['id']}"):
        st.write(c["text"])  # the actual chunk
```

Same idea in React (Session 7 app): split the answer on `/\\(\\d+)\\`, render `<sup>` chips, show the passage in a popover onClick.



Desi analogy: The senior consultant habit: “Revenue fell 12% — page 47 of the annual report.” Nobody argues with a man who carries page numbers. Your app should be that consultant.

What the user sees

Fire damage is covered up to Rs. 5,00,000 ^[1], but water-logging has a 2-year waiting period ^[2].

[1] home_policy_c2

“...fire and allied perils are covered up to Rs. 5,00,000 per policy year...”

[2] home_policy_c5

“...water-logging claims are admissible only after 24 months of continuous cover...”

Why users trust this instantly

Every claim is one click away from its evidence. The user can audit the AI — nobody audits a bare paragraph.



An honest badge — and a graceful “not found” path

- **Signal 1 — retrieval scores:** if even the BEST chunk is far from the question, the book probably doesn't cover it.
- **Signal 2 — model self-assessment:** ask “is the context sufficient? YES/PARTIAL/NO” along with the answer.
- Combine into one small badge. Never show raw cosine distances to users.

```
def confidence(distances):    # Chroma: lower = closer
    best = min(distances)
    if best < 0.35: return "High"
    if best < 0.60: return "Medium"
    return "Low"    # tune thresholds on YOUR corpus
```

Tune the two thresholds by eyeballing 20 real queries against your own corpus — they are not universal constants.

High

Best chunk very close + model says context sufficient. Show answer confidently.

Medium

Partial match. Show answer + “please verify with the cited source”.

Low

Nothing close in the corpus. Do NOT answer from model memory.

The empty answer is a FEATURE

“I could not find this in the policy documents” + a link to raise a ticket beats a beautiful hallucination every single time. Compliance teams will love you for it.



Desi analogy: The weather app says “70% chance of rain” — and you carry an umbrella. Honest uncertainty you can act on beats fake certainty. A forecaster who always says “100% sunny” is useless the day it pours.



Faithfulness checks — Session 3's eval gate grows two asserts

```
# test_faithfulness.py — add to the Session 3 gate

def test_answer_is_grounded():
    ans, chunks = rag_answer("What is the fire cover limit?")
    verdict = judge_llm(f"""Context: {chunks}
Answer: {ans}
Is EVERY claim in the answer supported by the context?
Reply YES or NO.""")
    assert "YES" in verdict      # faithfulness gate

def test_out_of_corpus_refuses():
    ans, _ = rag_answer("Who won the IPL in 2024?")
    # refusing IS the correct behaviour
    assert "NOT_FOUND" in ans

def test_citations_exist():
    ans, _ = rag_answer("Is water-logging covered?")
    assert re.search(r"\[\d+\]", ans) # must cite
```

Why this gate matters

- Faithfulness = every claim traceable to a retrieved chunk. Groundedness in one word.
- **Run in CI like Session 3: prompt changed? Chunking changed? Gate must stay green.**
- Judge can be a small cheap model — checking is easier than writing.



Desi analogy: The external examiner: he doesn't rewrite your answer sheet, he cross-checks every line against the textbook before marks are released.



HANDS-ON LAB · 60 MINUTES

Build the honest consultant: answers with sources, or says “not found”.

Ingest 3 policy documents into Chroma → grounded /ask endpoint with citations → UI with citation chips + confidence badge → prove it refuses out-of-corpus questions.

Think of it as training a junior consultant: read the files, quote the page, and never bluff the client.



LAB · STEP 0

Setup — a tiny corpus you fully control

Create 3 synthetic policy files

- motor_policy.txt — own-damage cover Rs. 3,00,000, zero-dep add-on, 2 claims/year cap
- health_policy.txt — Rs. 5,00,000 sum insured, 36-month waiting for pre-existing
- home_policy.txt — fire cover Rs. 5,00,000, water-logging after 24 months
- **Give each DISTINCT numbers — so you can verify retrieval picked the right doc**

```
# one-time setup (CPU is fine today)
python -m venv .venv && source .venv/bin/activate
pip install chromadb sentence-transformers
pip install litellm fastapi uvicorn streamlit

export OPENAI_API_KEY=... # or your S7 gateway
```

What you will hand in (60 min)

- ☐ 10 min — 3 policy docs written & ingested into Chroma
- ☐ 15 min — /ask endpoint: grounded prompt, citations, scores
- ☐ 20 min — UI: inline citation chips + confidence badge
- ☐ 10 min — out-of-corpus test: honest “not found” verified
- ☐ 5 min — the 3 faithfulness asserts run green



LAB · STEP 1

Ingest the corpus into Chroma (10 min)

Task: run ingest.py (from the walkthrough) on your 3 policy files.

```
python ingest.py
# -> 3 docs, ~12-18 chunks stored
```

```
# talk to the store directly – no LLM yet
python - <<'EOF'
import chromadb
col = chromadb.PersistentClient("./vectordb") \
    .get_collection("policies")
print(col.count(), "chunks")
r = col.query(query_texts=["fire damage cover"],
              n_results=2)
print(r["ids"], r["distances"])
EOF
```

While it runs, observe

- First run downloads the MiniLM embedding model (~90 MB) — one-time cost, like the first tiffin delivery learning the route.
- Query “fire damage cover” — did home_policy chunks come first? That's retrieval working, no LLM involved.
- Note the distance numbers for a GOOD match — you'll use them to tune the confidence thresholds.
- Now query “cricket score” — see how far the distances get. That gap IS your Low-confidence signal.



The grounded /ask endpoint (15 min)

Task: expose retrieve → ground → generate → cite as POST /ask.

```
# api.py – the response contract
@app.post("/ask")
def ask(q: Question):
    res = col.query(query_texts=[q.text],
                    n_results=4)
    answer = grounded_answer(q.text, res) # slide 12
    return {
        "answer": answer,                # contains [1][2]
        "citations": [
            {"n": i+1, "id": cid, "text": doc}
            for i, (cid, doc) in enumerate(zip(
                res["ids"][0], res["documents"][0]))],
        "confidence": confidence(
            res["distances"][0]),
    }
```

Test it before any UI

```
curl -X POST localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"text": "What is the fire
      cover limit?"}'
```

- Does the answer carry [1]-style markers? If not, tighten the system prompt — the model obeys formats it is SHOWN, so include a one-line example.
- Are citation ids real chunk ids (home_policy_c2)? Those come from ingestion — slide 11's ids.
- Is confidence “High” for this in-corpus question?



Citations + confidence badge in the UI (20 min)

Task: render the /ask response — chips, sources, badge. Streamlit OR your Session 7 Next.js app.

```
# app_ui.py — run: streamlit run app_ui.py
import re, requests, streamlit as st

q = st.text_input("Ask about your policies")
if q:
    r = requests.post(API + "/ask",
                      json={"text": q}).json()
    colour = {"High": "green", "Medium":
              "orange", "Low": "red"}[r["confidence"]]
    st.markdown(f":{colour}• {r['confidence']} confidence")
    st.markdown(chipify(r["answer"])) # slide 13
    for c in r["citations"]:
        with st.expander(f"[{c['n']}] {c['id']}"):
            st.write(c["text"])
```

Judge your own UX

- Ask “What is the health sum insured?” — chip [1] should open a health_policy chunk that CONTAINS Rs. 5,00,000. Click and check.
- Badge showing High for everything? Your thresholds are too loose — retune with the Step 1 distances.
- Show a colleague: can they verify an answer in under 10 seconds using only the chips? That's the bar.
- Next.js pairs: same JSON, render <sup> chips + a popover — the API contract doesn't change.



The honesty test: ask what the corpus cannot answer (10 min)

Task: ask questions your 3 documents can NOT answer — and watch what your app does.

```
# out-of-corpus probes
"Who won the IPL in 2024?"
"What is the premium for cyber insurance?"
"Does my policy cover treatment abroad?"
# (cyber & abroad are NOT in your 3 docs)
```

PASS looks like

"I could not find this in the policy documents." • badge = Low • no citations • optionally: "raise a ticket / contact your advisor".

If it hallucinates instead...

- Check 1 — system prompt: is "answer ONLY from context" + NOT_FOUND rule actually in the message you send?
- Check 2 — are you skipping the LLM call when confidence is Low? Cheapest fix: low confidence → return the not-found message directly.
- Check 3 — is some fallback path answering without retrieval (an old Session 7 route)?
- Then add the refusal assert from the eval slide so this NEVER regresses in CI.

Demo this refusal to the class — it earns more trust than any correct answer.



PITFALLS

Common RAG mistakes (learn from others' bruises)



Chunks too big or too small

2000-token chunks retrieve mush; 50-token chunks tear clauses apart. Start 300–500 with overlap, tune with real queries.



No citations

An answer nobody can verify is an answer nobody should trust. Numbered context + [n] markers cost you one prompt line.



Dumping ALL retrieved text

Top-k means k best — but check the scores. Stuffing 10 barely-related chunks buries the 2 good ones (raitā phail gaya).



Fake confidence — always “High”

A badge that never says Low is decoration, not information. Tune thresholds on real distances or drop the badge.



Answering out-of-corpus from memory

The model “helpfully” fills gaps with training data. That's a closed-book answer wearing an open-book costume.



Never evaluating faithfulness

Prompt tweak, new chunking, model upgrade — any of these can silently break grounding. Gate it in CI (Session 3!).



SESSION 8 · RECAP

What you can now do



See, not just read

Images + PDFs into Claude / GPT-4o / Gemini via one OpenAI-format message (LiteLLM, Session 7).



Extract with a gate

Photo → “extract as JSON” → Pydantic validation → ask-again UX when the photo is bad (Session 6).



Run the RAG pipeline

Ingest → chunk → embed → store → retrieve → ground → generate → cite. Open book, page numbers.



Ship honest UX

Inline [1][2] citation chips, source passages one click away, High/Medium/Low confidence badge.



Refuse gracefully

Out-of-corpus → “not found in the documents” — and a CI faithfulness gate so it stays that way.

Next session

Giving the App Hands: Tool Use with MCP

Today the app can SEE and QUOTE. Next it will ACT: a single agent that calls tools — search, database, ticketing — through the Model Context Protocol, safely.

Bring today's RAG app — Session 9 turns it into a tool your agent can call.